Journal of Systems Architecture 123 (2022) 102341

Contents lists available at ScienceDirect

Journal of Systems Architecture

journal homepage: www.elsevier.com/locate/sysarc

Autoware_Perf: A tracing and performance analysis framework for ROS 2 applications

Zihang Li [a,*], Atsushi Hasegawa [b], Takuya Azumi [a]

[a] Graduate School of Science and Engineering, Saitama University, Japan
[b] Research Institute of Systems Planning, Inc., Japan

ARTICLE INFO

Keywords:
ROS 2
Autoware.Auto
Performance evaluation
End-to-end latency
Linux trace tool

ABSTRACT

This paper proposes Autoware_Perf, a tracing and performance analysis framework for ROS 2 applications, such as Autoware.Auto. According to our survey, ROS 2 Tracing is an expected trace tool for measuring Autoware.Auto. However, ROS 2 Tracing can measure only the callback execution time of ROS 2 nodes running in the current system; it cannot measure node execution time and cannot measure yet the inter-node communication latency. Here we provide an improvement that arbitrarily selects the nodes for ROS 2 Tracing and provide a framework, which is based on ROS 2 Tracing, to measure the end-to-end latency of Autoware.Auto. Using this framework, we analyzed the trace data of ROS 2 systems generated by ROS 2 Tracing packages, and visualized the trace data as a graph. Autoware_Perf facilitates real-time performance evaluations but cannot measure yet all types of messages. Finally, we measure Autoware.Auto by improved ROS 2 Tracing and measure the end-to-end latency of Autoware.Auto by the proposed framework.

1. Introduction

As technology booms, self-driving systems have gained in popularity. An autonomous vehicle includes an advanced driver-assistance system (ADAS) that captures the surrounding information, ensuring safety, driver comfort, and complete control of the vehicle. ADAS can reduce the frequency of traffic accidents caused by distracted drivers who cannot adequately respond to changing traffic lights. Meanwhile, a rapid-response driving automation system that responds to the surrounding environment faster than humans remains to be developed. While a human needs at least 300 ms to perceive a new traffic situation and to react accordingly, this gives a maximum boundary of about 100 ms for detecting objects and a proper reaction by the automation system [1]. For this reason, real-time performance evaluation has become an important part of the development of high performance self-driving systems.

Recently, open-source self-driving stacks such as Autoware [2,3], and Apollo [4] have attracted attention, as they promise solutions to solve real-world problems. The first Autoware version was Autoware.AI based on ROS 1, which lacks real-time performance and cannot meet the real-time requirements of self-driving systems. These problems have been improved in ROS 2. The next version of Autoware.AI, Autoware.Auto based on ROS 2, follows best practices in modern software engineering and includes code reviews, continuous integration testing, thorough documentation, development guides, and a well-defined release process [5].

A Linux trace tool, named ROS 2 Tracing, evaluates the real-time performance of ROS 2 and is expected to measure the end-to-end latency of Autoware.Auto, including the execution time of a series of nodes and the communication latency between nodes. The node is the execution unit of ROS 2. The end-to-end latency defines the time between the receipt of LiDAR data and the steering angle calculations and speed results. In addition, an analysis tool is used to plot the trace data on a graph for visualization.

ROS 2 Tracing has several limitations: (1) the tool can only measure the execution time of callback (minimal schedulable entity in ROS 2); (2) it measures the callback duration (execution time of callback) of all nodes running at the same time and cannot select a specified node which is problematic for analysis and graph drawing.

This paper proposes the Autoware_Perf, an Autoware.Auto measurement and analysis framework. Autoware_Perf provides the following solutions to the above problems: (1) obtain the information of nodes from trace data and calculate the latency of Autoware.Auto; (2) optimize the visualization analysis method for the latency.

Contributions: Autoware_Perf offers the following advantages over the existing tool for Autoware.Auto latency measurement:

- `Selective measurement:` The selective functionality is designed for ROS 2 Tracing and Autoware_Perf can generate an architecture file that has the information of nodes to selectively

* Corresponding author.
E-mail address: li.z.691@ms.saitama-u.ac.jp (Z. Li).

https://doi.org/10.1016/j.sysarc.2021.102341
Received 27 October 2021; Accepted 13 November 2021
Available online 20 December 2021
1383-7621/© 2021 Elsevier B.V. All rights reserved.

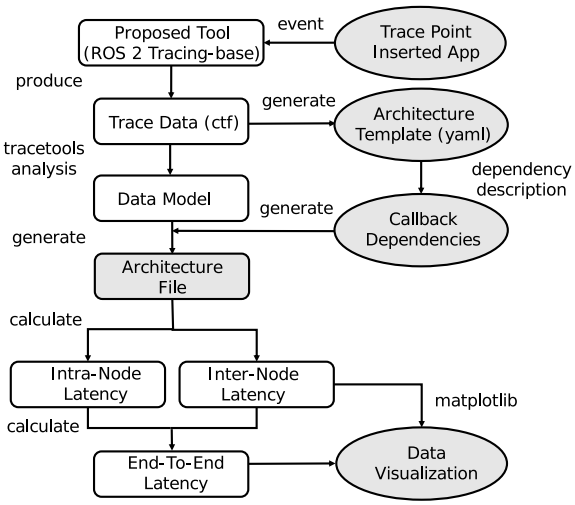


Fig. 1. Flowchart of the system model.

measure any parts of Autoware.Auto. Therefore, it can find bottlenecks in Autoware.Auto.

- `Trace point insertion:` Autoware_Perf can measure the communication latency between nodes by inserting trace point into the ROS 2 rclcpp (ROS Client Library for C++ package), which is a new functionality of ROS 2 Tracing.
- `Overall measurement:` Autoware_Perf can measure the latency of any path for analyzing the end-to-end latency from the entry node to the end node. Developers can use Autoware_Perf to determine whether the ROS 2 application meets expectations.

Organization: The remainder of this paper is organized as follows. Section 2 describes the system model for data analysis based on ROS 2 Tracing and its additional functionalities. Section 3 discusses the problems when tracing Autoware.Auto with the trace tool and proposes their solutions. Section 4 evaluates Autoware_Perf in Autoware.Auto end-to-end latency measurement. Related work is discussed in Section 5, and conclusions are presented in Section 6.

2. System model

This section describes the system model (see Fig. 1). To follow the measurements, the user runs the software and proposed framework at the same time. The trace result is output as a common trace format (ctf) file. The ctf file is in binary trace format, which allows the native generation of traces by any C/C++ application or system [6]. In addition, the framework generates a yaml file that describes the application architecture and callback dependencies. Finally, by reading the trace data and architecture file, Autoware_Perf calculates the intra-node/inter-node/end-to-end latency.

This section first introduces the important parts of self-driving systems (ROS and Autoware) and investigates the trace tools for ROS 2 latency measurement. Finally, it introduces the tool selected for tracing Autoware.Auto.

2.1. Robot operating system

ROS 1 is a suite of software frameworks providing middleware for robot software development. Sets of executing ROS-based processes are represented as nodes. The nodes may receive, post, and multiplex sensor data and messages for control, state, planning, actuation, and others. Despite the importance of reactivity and low latency in robot control, ROS 1 is not a real-time operating system (RTOS). ROS 2 is integrated with real-time code to provide support for real-time systems, which is lacking in ROS 1 [7].

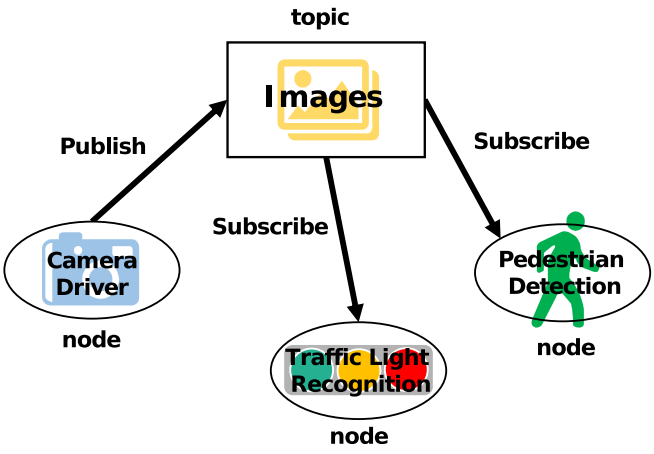


Fig. 2. Communication mechanism of ROS.

The new use cases supported by ROS 2 are real-time systems, small embedded platforms (e.g., sensor nodes), non-ideal networks. To satisfy the requirements of these new use cases, the existing version of ROS 1 will be reconstructed to improve the application programming interface and will incorporate new technologies such as the data distribution service (DDS), which will replace ROS 1 transport system. DDS is an industry-standard real-time communication system and path middleware providing reliable publish/subscribe transport similar to that of ROS 1 [8].

ROS 2 dissociates complex systems into many modular nodes for single module purpose. Each node can send and receive data to other nodes via topic by using publish/subscribe communication. For example, as shown in Fig. 2 [2], the "Camera Driver" node sends messages to the "Images" topic, and the messages in the topic are received by the "Traffic Light Recognition" and "Pedestrian Detection".

2.2. Autoware.Auto

Autoware is designed for autonomous vehicles driving through urban areas. To adapt Autoware to freeways and highways, additional modules might be required. The supported modules in Autoware are listed below [2]:

- `Sensing:` When recognizing a road environment, Autoware is usually assisted by LiDAR scanners and cameras. LiDAR scanners measure the distance to objects by illuminating a target with pulsed lasers and measuring the time of the reflected pulses, whereas cameras recognize traffic lights and extract additional features of the scanned objects.
- `Computing:` Computing intelligence is an essential component of autonomous vehicles. The vehicle computes the final trajectory from sensor data and 3D maps and communicates with the actuation modules.
- `Actuation:` Autonomous vehicles must follow the determined local trajectory. In path following, the actuation commands for the ego-vehicle are generated by a pure pursuit algorithm. In vehicle control, the target waypoint is continuously updated until the goal is reached.

With the development of self-driving applications, ROS 1 has matured, and ROS 2 version can now be productized. Autoware has also graduated from the research and prototyping phase, and its upcoming version, Autoware.Auto is being developed for wider commercial applications.

Table 1
Trace tool survey.

Tool	Measurable	Non-measurable
Apex.AI performance_test	ROS 2, DDS (Pub/sub): Latency, CPU usage, maximum RAM virtual page size, number of page faults, number of context switches	One topic cannot accommodate multiple publishers; large data causes internal loops
ROS 2 Tracing (using LTTng)	Analysis and visualization of trace data with other tools: Callback duration, message elapsed time, message size, memory usage, execution time, and the ratio of process nodes and components	DDS
Open Robotics buildfarm_ perf_tests (using Apex.AI)	RMW: Average latency, CPU utilization, number of send and receive packets per second, number of lost packets, maximum resident set size (RSS)	Latency between different RMW implementations
iRobot ros2_performance	Specified node name, topic name, and message structure; each element in a generated measurement node	Performance of customized nodes

2.3. Linux trace tool

Real-time robotic systems are difficult to analyze, as their internal processing continually responds to changing sensory inputs. Therefore, the data of robotic software must be continuously monitored and recorded to ensure their deterministic behavior, containment within the resource limits, and availability for later analysis. Runtime data can be routinely monitored by tracing, which is well established, already well integrated with operating systems, and incurs minimal overhead [9].

Here, we investigate the existing Linux tracing tools that measure the execution time and the communication latency of ROS 2. The summary is shown in Table 1.

Apex.AI performance_test is a test tool that can measure the latency of pub/sub based communication frameworks such as ROS 1 and DDS, but cannot measure a topic that has multiple publishers. Due to multiple publishers to a single topic that exists in ROS 2, Apex.AI cannot measure the end-to-end latency of ROS 2 [10]. LTTng can measure Linux kernel and user application [11]. ROS 2 Tracing, which is based on LTTng, can measure the latency of node callback duration, but cannot measure DDS directly [12]. Open Robotics can measure the latency of RMW (ROS Middleware Interface) but is not suitable for different RMW implementations. Due to different RMW implementations are always used in ROS 2, Open Robotics cannot measure the end-to-end latency of ROS 2 [13]. iRobot can measure specified nodes and topics, but cannot measure the customized nodes. Due to customized nodes exist in large numbers in Autoware.Auto, iRobot cannot be used [14]. Specifically, we consider the capabilities of these tools, and select the most suitable tool, LTTng and ROS 2 Tracing, in a comparison study. To trace ROS 2 nodes, we applied ROS 2 Tracing based on LTTng, which is developed by micro-ROS and can trace Linux kernels and user applications.

2.4. LTTng

LTTng (the next generation Linux Trace Toolkit) is an open-source software toolkit that simultaneously traces the Linux kernel, user applications, and user libraries. LTTng is deployed as a group of loadable kernel modules for kernel tracing and a user-space component for the management and consumption of tracing. LTTng is mainly distinguished by its ability to produce correlated kernel and user-space traces with low overhead. LTTng supports architectures such as IA-32 (x86), x86-64, PowerPC, Arm, and MIPS, amongst others.

2.5. ROS 2 Tracing

ROS 2 Tracing traces the runtime data of ROS 2 and provides analysis tools that plot the callback duration. ROS 2 Tracing traces nodes of ROS 2 using the trace functionality of the LTTng user application. LTTng can record trace points such as ROS events. The names, types, and timestamps of these events are provided in the trace data. The timestamps can be used in the callback duration calculations. The trace data are saved in ctf format, which can be converted into a human-readable version using the babeltrace command. The convert command converts a ctf trace into a file that can be read more quickly and the process command converts the file into a data model.

The analysis tool analyzes the trace results. By visualizing the data, we can efficiently analyze the performance of ROS 2. ROS 2 Tracing allows the recording of runtime data from the system, such as time of a message arrival, triggering of a timer, and time of running a callback, which are processed in the ROS 2 framework.

3. Design and implementation

This section describes the design and implementation of the proposed framework (Autoware_Perf). Autoware_Perf obtains information of nodes from trace data and calculates the latency of Autoware.Auto.

As done in compositional performance analysis (CPA), a complex ROS graph can be analyzed by computing the individual response-time upper bounds for each callback. Path latencies are then obtained by summing the individual response times of the callbacks. Unfortunately, none of the existing CPA instantiations can compute per-callback response times [15].

For this reason, the response time of ROS 2 callback cannot be calculated by CPA. Therefore, we obtain the latency of callback by Autoware_Perf rather than by theoretical calculation. Autoware_Perf includes a proposal for measuring the end-to-end latency of Autoware.Auto. In different subsections, this section discusses the functionality improvements and execution environment of Autoware_Perf: (1) Selective Functionality; (2) Calculation Functionality; (3) Architecture File; (4) Autoware.Auto running without ADE.

3.1. Selective Functionality

Autoware_Perf offers Selective Functionality of the callback function to measure the specific node. To measure the end-to-end latency of Autoware.Auto, we need to analyze which nodes are in the end-to-end path. According to our survey, ROS 2 tracing measures all callback functions of nodes and makes the analysis of nodes difficult. Therefore, we first distinguish the callback function by the node name.

The code that implements the Selective Functionality is added to the obtained data model in ROS 2 Tracing. The callback function of the specified node can then be obtained.

As shown in Fig. 3, the code segment that retrieves the information of nodes by the type names, is manually inserted into the original code. The callback_symbols save the instances of the callback function. The type_name variables store the callback types, such as subscription, timer, and service/client. In addition to type_name, the subscription and publisher have a node name and a topic name. The timer obtains the timer identification (tid) and timer period, and the service/client obtains the node and service names through get_callback_owner_info().

The function named get_XXX_handle_nodeinfo(), where XXX represents the callback type, retrieves the node information of different types through type_name. Note that the node information of timer, tid and period, cannot specify the node name in this function. Instead, get_node_names_from_tid() obtains the node name given the tid.

The python plot tool is used for visualization analysis. The visualization results are shown in Section 4.

```
#Select callback by typename

for obj, symbol in callback_symbols.items():
    owner_info = data_util.get_callback_owner_info(obj)
    type_name = owner_info.split("--")[0]
    if type_name == "Subscription":
        node_info = data_util.get_subscription_reference_nodeinfo(obj)
        node = list(node_info.values())
        node_name = node[0]
    elif type_name == "Timer":
        node_info = data_util.get_timer_handle_info_nodeinfo(obj)
        node = list(node_info.values())
        tid = node[0]
        node_name = data_util.get_node_names_from_tid(tid)
    elif type_name == "Service":
        node_info = data_util.get_publisher_handle_info_nodeinfo(obj)
        node = list(node_info.values())
        node_name = node[0]
    else:
        print("error")
...
#Select the callback from node named mpc_controller

if(node_name == "mpc_controller"):
    show(row(duration, hist))
else:
    continue
```

Fig. 3. Select callbacks by type_name.

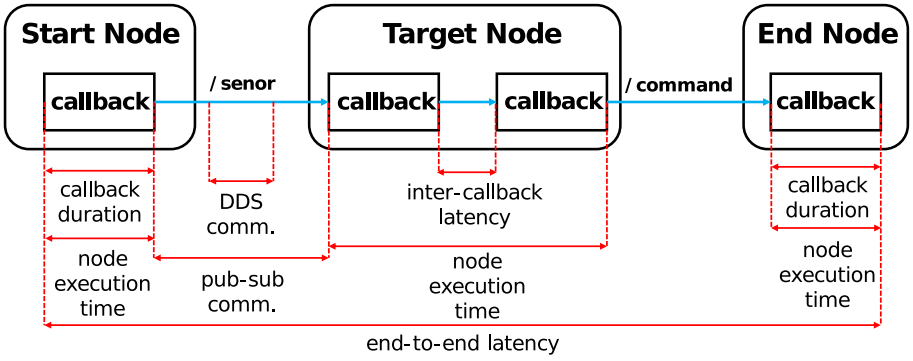


Fig. 4. End-to-end latency.

3.2. Calculation Functionality

Autoware_Perf is used to calculate the end-to-end latency, which is the sum of node execution time and publish/subscribe communication latency (communication latency between nodes), as shown in Fig. 4.

By using the Selective Functionality, Autoware_Perf can measure node execution time individually. The Selective Functionality is used to not only select a specific node in the calculation function, but also make the connection between nodes clearer. Therefore we can plot the node graph to analyze the end-to-end path.

Node execution time, which including callback duration and inter-callback latency, is from the start of the subscribe or timer callback execution to the end of the publish. Callback duration is from the start of callback to the end of callback. Inter-callback latency is from the end of callback execution to the start of callback execution. Here, inter-callback latency from the end of callback execution to the start of the next callback execution when a dependency exists between callbacks through member variables. If only one callback is executed in the path within a single node, inter-callback latency are not included. Publish/subscribe communication latency is from the end of node publish to the start of the subsequent callback execution.

To trace publish/subscribe communication, the trace point is inserted into the subscriber and publisher class in rclcpp repository (see Fig. 5) [16,17]. Any subscription callback is executed when a node receives a message from the topic. The trace point is inserted at the start time of the callback. During a publisher function execution, the trace point is inserted when the node sends a message to the topic, which marks the start time of publishing. The communication latency between nodes is then calculated by subtracting the publishing start time from the subscription-callback start time. The callback execution time is calculated by subtracting the start time from the end time of a callback.

In this case, we find Autoware.Auto nodes are not one input to one output relation, due to some messages lost when the messages are sent into subscription callback. The lost messages make calculation complex due to Autoware_Perf not being able to trace the specific message. Since the messages between different nodes are not one-to-one correspondence, ordinary addition operations are not suitable for the accumulation of node latency. Therefore, the end-to-end latency is calculated by the convolution integral of the discretized probability distribution. The convolution integral is the general method to calculate the sum of two random variables probability distribution if they are independent. System S1 with a latency probability distribution of P1 and system S2 with a latency probability distribution of P2 are obtained by normalizing a histogram of latency that can be calculated by the

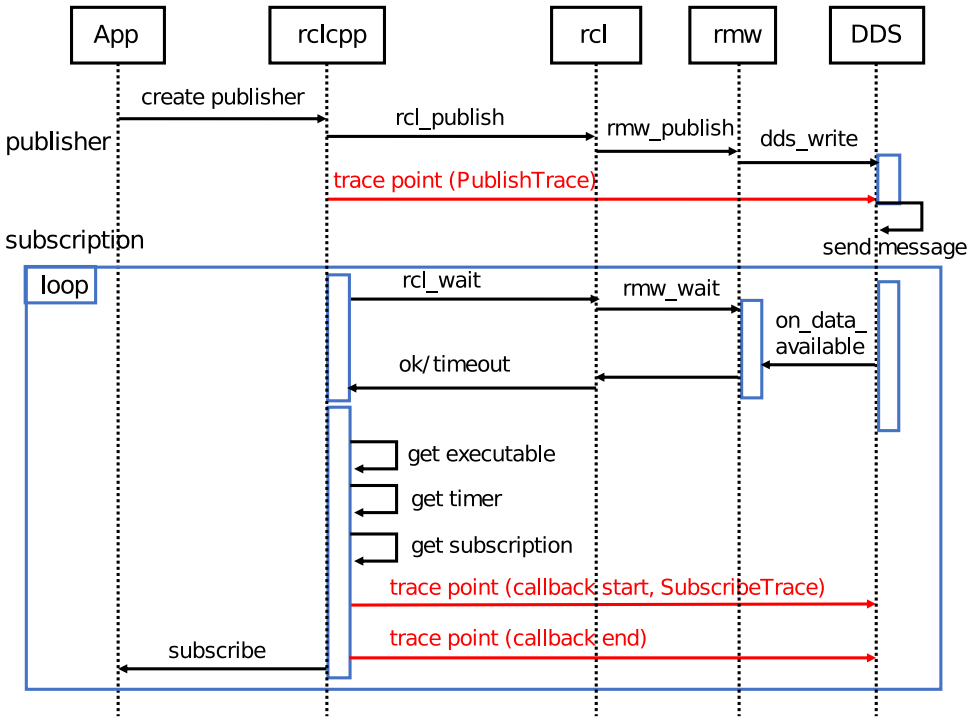


Fig. 5. Processing sequence of the added Calculation Functionality.

convolution integral. At this time, the sum of latency probability distribution of system 1 and system 2, is calculated by the following formula:

$$P(X) = \sum_{t=0}^{X-1} P1(t)\frac{1}{2}(P2(X-t) + P2(X-t-1)) \quad (1)$$

P is a convolution integral of P1 and P2. P1 and P2 show the probability of latency in the interval with a certain width. P(X) represents the probability that the latency falls in the interval X and t is an integer variable from 0 to X − 1. The probability distribution P is the convolution sum of system 1 and system 2 latency, that is, the estimated value of the sum of system 1 and system 2 latency.

To obtain the start and end times of a callback, the start and end events of the callback are traced by ROS 2 Tracing. The communication latency between the callbacks is obtained by subtracting the callback end time from the start time of the subsequent callback.

The trace data can be collected into an architecture file (architecture.json). To calculate the node execution time, we write callback dependencies in architecture.json, which includes the information of nodes, such as start and end nodes, callback name and type, published or subscribed topic names, and subsequent callback symbols (see Section 3.3). In particular, the topic, which does not identify the published callback, can be written into the unlinked publish the topic list. If the callbacks in the node are interdependent, the symbol name of the subsequent callback symbols is described. For simplicity, we simulate three plausible types of nodes in the path. The execution time of a no-dependency node (the simplest node type) depends only on the subscription callback function. The execution time of a sub-dependency node depends on both multiple subscription callbacks, and the latency between callbacks. Finally, the execution time of a timer-dependency node depends on the timer callback, subscription callback, and the wake-up time of the timer callback.

This framework uses a fork version of ROS 2 with the added performance measurement function. By separating from the main development of ROS 2 and making flexible changes to related packages, it has the following features.

- Performance can be measured with any rclcpp application.
- ROS 2 Tracing is used for base tracing tool:
  - Low overhead trace
  - Kernel and user-space events can be traced at the same time
- Bottlenecks can be found for the following measurement targets:
  - Callback execution time
  - Latency between callbacks with mutual dependency
  - Communication latency between nodes
  - Node execution time
  - End-to-end latency

3.3. Architecture File

The architecture file, which can be obtained from trace data, is a file that describes the information required to calculate the end-to-end latency. However, a few manual modifications in the architecture file are required to calculate the end-to-end latency. The architecture file format is shown in Fig. 6, and the manual modifications are commented. In addition, the architecture file can be plotted to a node graph by using Graphviz.

3.4. Autoware.Auto running without ADE

Autoware.Auto generally runs in the ADE environment, a modular Docker-based tool that provides a common, consistent development environment for all developers. Since LTTng is not installed in the ADE environment, this will cause some trouble for our experiments, such as installing our framework every time when entering ADE. For this reason, we build and run Autoware.Auto without ADE [18] and performed tracing in the local environment.

4. Evaluation

To verify the added functionalities (Selective Functionality and Calculation Functionality) of the previous section, we measure Autoware.Auto using Autoware_Perf. The proposed method is developed

```
{ "target_path": {
    "start_node_name":   # The start node's name
    "end_node_name":    # The end node's name  },
  "nodes": [
  {  "name":         # name of the node,
     "callback_dependency": {
     "Callback_symbol_name":  # The subsequent callback symbol name},
     "publish": {
     "Topic_name":    # The callback that publishes to the topic},
     "callbacks": [
     { "type":          # Callback type (timer_callback),
       "period":        # Timer period (ns),
       "symbol":        # Callback symbol name },
     { "type":          # Callback type (subscribe_callback),
       "topic_name":  # topic name to subscribe,
       "symbol":        # Callback symbol name } ] } ] }
```

Fig. 6. Architecture file.

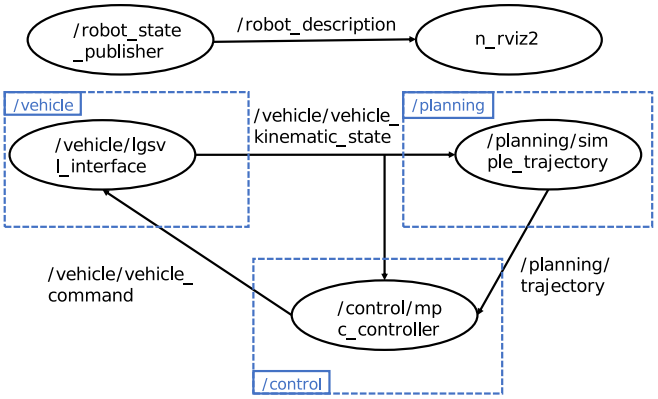


Fig. 7. Trajectory following nodes of Autoware.Auto.

based on ROS 2 Tracing, and ROS 2 Tracing is developed based on LTTng. The evaluation environment is Ubuntu 20.04 on a laptop computer, which has Intel Core i7-9750H CPU and NVIDIA GeForce GTX 1660 Ti GPU. The version of Autoware.Auto is 1.0.0 and the version of ROS 2 is foxy, which is evaluated by Autoware_Perf.

4.1. Selective Functionality

The selected functionality addition was evaluated on the trajectory following test nodes by improved ROS 2 Tracing. The trajectory following test nodes is a series of nodes, which generate and visualize a trajectory. The communication flow of the trajectory following test nodes is shown in Fig. 7. The vehicle node sends the vehicle kinematic state to the planning and the control nodes. The plan node sends the plan (next action) to the control node. Finally, the control node sends the commands that govern the operation of the vehicle according to the vehicle kinematic state and plan.

The improved trace tool arbitrarily measures the callback duration at the nodes. The timestamp and duration, indicating the start time and execution time of the callback, respectively, are returned by the get_callback_durations() function, and the durations of different callbacks are plotted [19]. The callbacks of the "mpc_controller" node subscribed to different topics are selected and the callbacks execution times are plotted as shown in Fig. 8.

As shown in Fig. 8(a) and (b), the execution times of the callback received messages on the tracking of the robot coordinate frames over time are plotted. Fig. 8(c) shows the callback handling of the vehicle state. As the trajectory of the vehicle tracks and the vehicle position continuously alters, the state-related topic messages are constantly changing over time, and the callback function is repeatedly executed. Fig. 8(d) shows the parameter setting of this node, which executes in the initial stage.

4.2. Calculation Functionality

An end-to-end latency of Autoware.Auto was measured by Autoware_Perf. The node path is an Autonomous Valet Parking (AVP) demonstration, which uses Autoware.Auto to provide a valet parking service. The overview and installation instructions can be referred to this page: [20]. The goal is to direct the car to autonomously park in a parking lot, and the end-to-end path for the AVP demonstration is shown in Fig. 9.

We run it in the LGSVL simulator and trace it by using **ros2 trace** command. To analyze the trace data, we use the architecture file and find the end-to-end path, which is shown in red in Fig. 9. The start time of the end-to-end path is defined as the time when the Point Cloud Fusion node publishes the fused point cloud data topic, and the end time is defined as the time when the MPC controller node publishes vehicle control instructions. All of the nodes' latency in the end-to-end path is measured, and two of the nodes latency are shown in Fig. 10 as an example. All of the inter-node communication latency is measured, and two of the inter-node communication latency are shown in Fig. 11 as an example. The left figure is the inter-node communication latency from the point cloud node to the voxel grid cloud node, and the right figure is

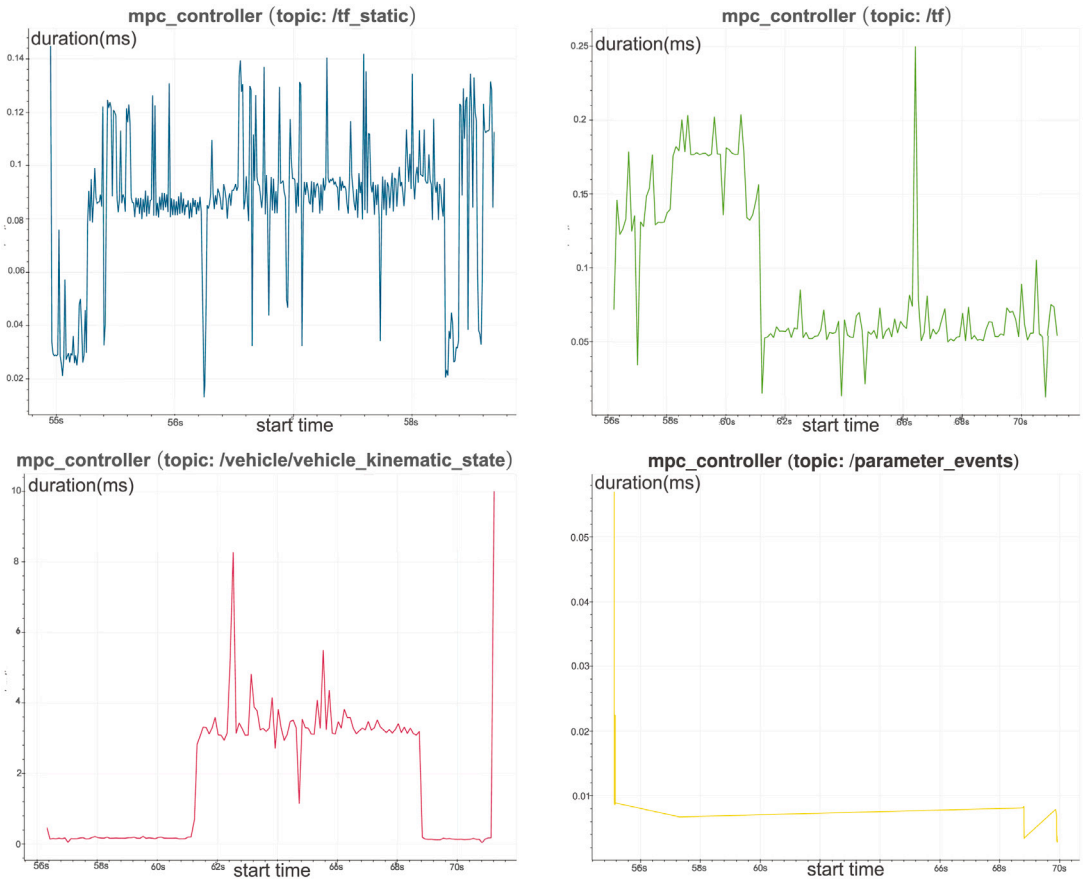


Fig. 8. Callback execution times of the mpc_controller node for different topics.

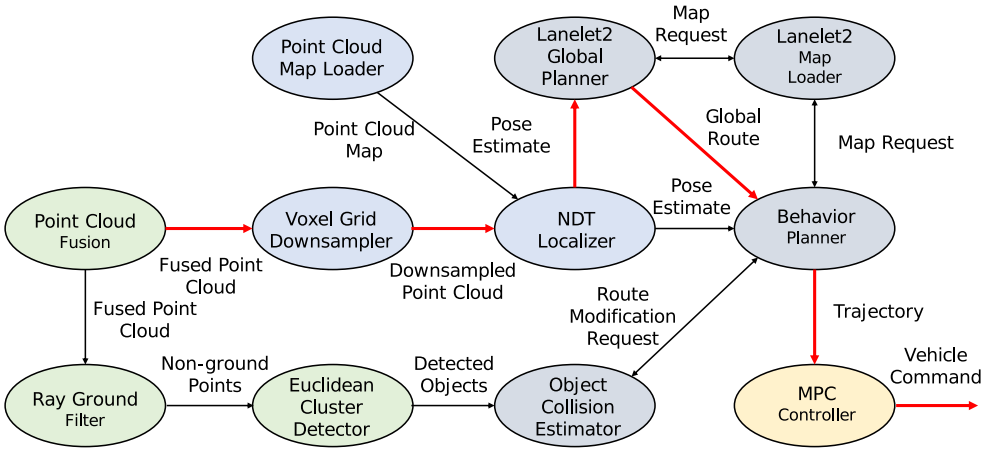


Fig. 9. AVP demonstration. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

the inter-node communication latency from the behavior planner node to the MPC controller node. Finally, the end-to-end latency is calculated by a convolution integral, which is shown in Fig. 12.

We can see that the end-to-end latency of the system is basically distributed within 50 ms, which has reached the current standard for safe driving (100 ms). The end-to-end latency is 93% within 100 ms, and the maximum value exceeds 200 ms. This means that the autonomous driving system still needs to be improved, especially the p2d ndt localizer node. In future work, we will consider verifying this framework by using other frameworks that may appear in the future or theoretical value calculation for comparison.

4.3. Lessons Learned

In this paper, we investigated Linux trace tools and proposed a framework to trace the latency of Autoware.Auto. The trace data generated by ROS 2 Tracing provided information on ROS events. Among this information, the timestamps of the ROS events were valuable

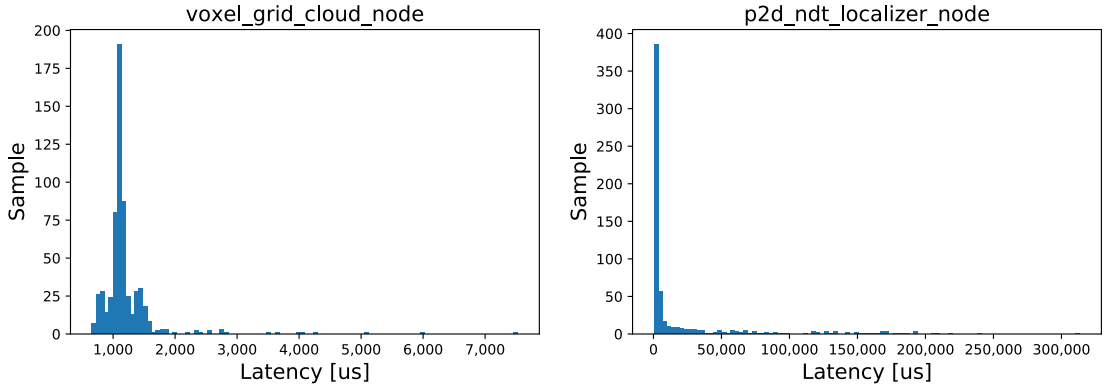


Fig. 10. Node execution time.

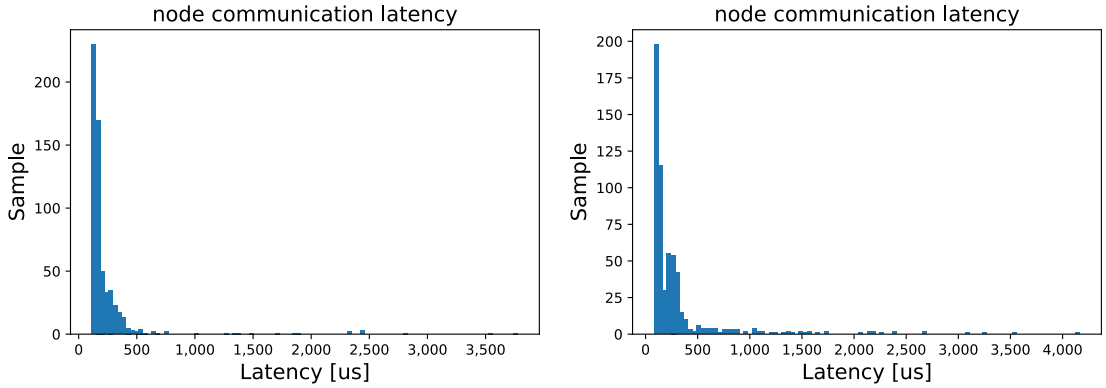


Fig. 11. Communication latency between nodes.

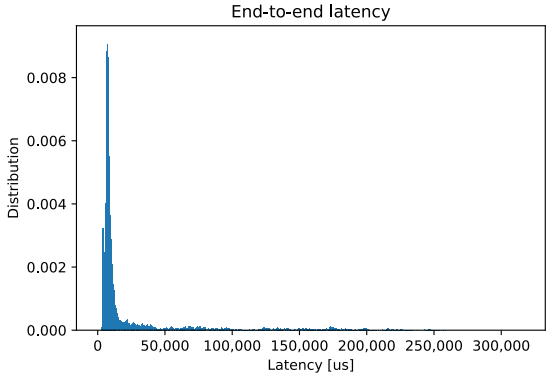


Fig. 12. End-to-end latency of Autoware.Auto.

for calculating the end-to-end latency. The latency durations were visualized using the python plot tool. However, ROS 2 Tracing has limited ability to trace ROS 2 applications.

First, ROS 2 Tracing measures only the callback duration of running ROS 2 nodes and lacks provision for individual node selection. To avoid this limitation, we obtain the callback duration at specified nodes by inserting a selective function into the analysis tool for ROS 2 Tracing.

Second, ROS 2 Tracing cannot measure the latency between nodes or callbacks. For this reason, we propose a framework based on ROS 2 Tracing for ROS 2 and ROS 2 applications. Autoware_Perf adds trace points into ROS 2 rclcpp, increasing the number of traceable ROS events. The latency between nodes can be measured for calculating the end-to-end latency.

At present, Autoware_Perf cannot specify the lost messages. To trace Autoware.Auto and other ROS 2 applications, we anticipate an improved framework without this restriction.

5. Related work

In the present study, the latency of ROS 1 and ROS 2 was measured and analyzed. ROS 1 is mainly used on robots for research purposes and provides many pre-implemented feature packages, whereas DDS has an extensive set of Quality-of-Service settings. Profanter et al. [21] evaluated ROS 1 and DDS. The performance comparison shows that DDS delivers high performance, whereas ROS 1 shows a significant slowdown in the round-trip time of packages sent to the server.

A real-time scheduling framework for ROS 1 called ROSCH [22] was presented by Saito et al. and the real-time performance of ROSCH was evaluated to ensure that the end-to-end latency is guaranteed to be less than a certain value. To reduce the overhead of ROS 1 and ROS 2 IPC, Wang et al. [23] proposed an efficient IPC technique called TZC. To compare the reliability, they count the number of messages received by each subscriber. The success rates of message reception are relatively high using TZC in ROS 2, subscribers can always receive all messages due to the default QoS configuration of DDS making ROS 2 more reliable than ROS 1 when transmitting small messages.

Tracing is an analytical method that continuously records real-time system run-time data. As the requirements of real-time applications and constraints increase under security concerns, Linux is being ported to real-time environments. The development of PREEMPT_RT real-time

Table 2
Comparison of related work.

	ROS 1/ROS 2	ROS 2 application	Latency analysis	End-to-end latency	Trace point	Customized analysis
ROS 1 performance analysis [21]	✓		✓			
ROSCH [22]	✓		✓			
ROS 2 performance analysis [8,23]	✓		✓			
ROS-lite [3]	✓		✓			
Self-driving architecture [1]	✓	✓	✓			
Autoware on board [2]	✓		✓	✓		
Bottlenecks in self-driving systems [24]		✓	✓	✓	✓	
ROS 2 response-time analysis [15]	✓		✓	✓		
ROS 2 real-time executor [17]	✓		✓			
PREEMPT_RT [25]			✓		✓	
Linux tracer survey [26]			✓		✓	
Apex.AI performance_test [10]	✓	✓	✓			
Open Robotics buildfarm_perf_tests [13]	✓	✓	✓			
iRobot ros2_performance [14]	✓	✓	✓			✓
ROS 2 Tracing [12]	✓	✓	✓		✓	✓
This paper	✓	✓	✓	✓	✓	✓

patch [25] can reduce Linux kernel latency. The migration from general OS to RTOS should improve kernel latency and jitter by performance analysis.

Trace tools on Linux systems can measure the latency of the real-time application. Gebai et al. [26] measured and compared the tracing overheads of various trace tools on Linux systems. Among many trace tools, LTTng and Ftrace exhibited the lowest overhead. In addition, LTTng is easy to use and feature-rich. For example, LTTng is compatible with analytical tools that understand the ctf output format.

Trace tool was rarely considered in the past research. Kato et al. [2] compared the performance of Autoware.AI on the DRIVE PX2 platform and a high-end computer. The performances of these packages are evaluated in an end-to-end latency as the total completion time of a specified set of packages during running. The end-to-end latency of each test package was measured through the timespec_get function, which assesses the difference between the start and end times of the corresponding process. The worst execution time was approximately three times higher on the DRIVE PX2 platform than on the laptop computer. Their observation is that this difference is caused by the performance of CPU cores rather than the GPUs. Based on the above results, the DRIVE PX2 platform provides the acceptable level of performance for Autoware installed to real autonomous vehicles, if the parameters of each functional node are configured appropriately.

Understanding the sources of latency and energy consumption is key in order to improve self-driving systems. Becker et al. [24] measured single node execution time, and the perception end-to-end latency of Autoware.AI. To measure the perception end-to-end latency, they traced the timestamp of messages since messages first entered the system until messages were published by the final perception nodes. Their measurements show that the end-to-end latency (worst case among all computation paths) always reaches more than 200 ms. Thereby, there are moments during the AV driving where the system takes more than twice the time commonly stipulated to drive safely (100 ms). Notwithstanding, they used a high-end computer, and larger computing arrangements are hardly an option for AVs due to size and energy consumption constraints. This indicates self-driving algorithms and computing platforms must be improved even further to assure the required performance.

The related work of ROS 1 or ROS 2 latency measurement is briefly summarized in Table 2. Compared to the related work, this paper is the first paper to measure the end-to-end latency of Autoware.Auto.

6. Conclusion

This paper proposed Autoware_Perf, a tracing and performance analysis framework for ROS 2 applications, such as Autoware.Auto. Autoware_Perf can measure the latency of each part in Autoware.Auto. We analyzed the performance of self-driving software. After finding no tool for identifying and measuring the end-to-end latency of Autoware.Auto in the existing research, we designed a framework that measures the end-to-end latency, including the node execution time and the communication latency between nodes. In the present study, the performance of ROS 2 was evaluated in terms of the end-to-end latency. To identify performance bottlenecks, our framework can also measure ROS 2 intra-node and inter-node latency. Autoware_Perf is all open-source, and can be downloaded from Github https://github.com/azu-lab/ROS2-E2E-Evaluation.

In future work, we will consider solving the problem that the proposed framework cannot measure and analyze the ROS 2 service communication method, which is a call-and-response communication method instead of the publish/subscribe communication method.After solving the problem about service, we will consider measuring more complex paths, such as multiple publisher node to subscriber node. We are expected to calculate much end-to-end latency in different paths. By comparing the latency, we may easily find the performance bottlenecks in the system. In addition, we may consider designing a graphical user interface through which users can analyze the ROS 2 applications without understanding the internal code or writing instructions.

Acknowledgments

This work was partially supported by the New Energy and Industrial Technology Development Organization (NEDO).

References

[1] M. Reke, D. Peter, J. Schulte-Tigges, S. Schiffer, A. Ferrein, T. Walter, D. Matheis, A self-driving car architecture in ROS 2, in: Proc. of International SAUPEC/RobMech/PRASA Conference, 2020.
[2] S. Kato, S. Tokunaga, Y. Maruyama, S. Maeda, M. Hirabayashi, Y. Kitsukawa, A. Monrroy, T. Ando, Y. Fujii, T. Azumi, Autoware on board: enabling autonomous vehicles with embedded systems, in: Proc. of ACM/IEEE International Conference on Cyber-Physical Systems (ICCPS), 2018.
[3] T. Azumi, Y. Maruyama, S. Kato, ROS-lite: ROS framework for NoC-based embedded many-core platform, in: Proc. of IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2020.
[4] Apollo, Baidu's Open Autonomous Driving Platform, 2018, URL: https://www.marklines.com/en/report_all/rep1787_201811.
[5] Autoware.Auto/Introduction, 2021, URL: https://autowarefoundation.gitlab.io/autoware.auto/AutowareAuto/.
[6] Common Trace Format, 2008, URL: https://diamon.org/ctf/.
[7] Robot Operating System Wikipedia, 2021, URL: https://ja.wikipedia.org/wiki/Robot_Operating_System.
[8] Y. Maruyama, S. Kato, T. Azumi, Exploring the performance of ROS 2, in: Proc. of International Conference on Embedded Software (EMSOFT), 2016.
[9] Getting started with ROS 2 tracing, 2021, URL: https://micro-ros.github.io/docs/tutorials/advanced/tracing/.
[10] Apex.AI: performance test, 2021, URL: https://gitlab.com/ApexAI/performance_test/.
[11] The LTTng Documentation, 2021, URL: https://lttng.org/docs/v2.12/.
[12] ROS 2 tracing, 2021, URL: https://gitlab.com/ros-tracing/ros2_tracing.

[13] Open Robotics: buildfarm perf tests, 2021, URL: https://github.com/ros2/buildfarm_perf_tests.
[14] iRobot: ROS2 performance, 2021, URL: https://github.com/irobot-ros/ros2-performance.
[15] D. Casini, T. Blaß, I. Lütkebohle, B.B. Brandenburg, Response-time analysis of ROS 2 processing chains under reservation-based scheduling, in: Proc. of Euromicro Conference on Real-Time Systems (ECRTS), 2019.
[16] Real-Time Executor, 2021, URL: https://micro-ros.github.io/docs/concepts/client_library/real-time_executor/.
[17] Y. Yang, T. Azumi, Exploring real-time executor on ROS 2, in: Proc. of IEEE International Conference on Embedded Software and Systems (ICESS), 2020.
[18] Installation w/o ADE, 2021, URL: https://autowarefoundation.gitlab.io/autoware.auto/AutowareAuto/installation-no-ade.html.
[19] ROS 2, 2020, tracing/tracetools_analysis/analysis/callback_duration.ipynb URL: https://gitlab.com/ros-tracing/tracetools_analysis/-/blob/master/tracetools_analysis/analysis/callback_duration.ipynb.
[20] Autoware.Auto: Autonomous Valet Parking Demonstration, 2021, URL: https://autowarefoundation.gitlab.io/autoware.auto/AutowareAuto/avpdemo.html.
[21] S. Profanter, A. Tekat, K. Dorofeev, M. Rickert, A. Knoll, OPC UA versus ROS, DDS, and MQTT: Performance evaluation of industry 4.0 protocols, in: Proc. of IEEE International Conference on Industrial Technology (ICIT), 2019.
[22] Y. Saito, F. Sato, T. Azumi, S. Kato, N. Nishio, ROSCH: Real-time scheduling framework for ROS, in: Proc. of IEEE International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA), 2018.
[23] Y. Wang, W. Tan, X. Hu, D. Manocha, S. Hu, TZC: Efficient inter-process communication for robotics middleware with partial serialization, in: Proc. of IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2019.
[24] P.H.E. Becker, J.M. Arnau, A. González, Demystifying power and performance bottlenecks in autonomous driving systems, in: Proc. of IEEE International Symposium on Workload Characterization (IISWC), 2020.
[25] F. Reghenzani, G. Massari, W. Fornaciari, The real-time linux kernel: A survey on PREEMPT_RT, ACM Comput. Surv. (2019).
[26] M. Gebai, M.R. Dagenais, Survey and Analysis of Kernel and Userspace Tracers on Linux: Design, Implementation, and Overhead, ACM Comput. Surv. (2018).